

Transformer技术原理

Transformer 或者说所有大语言模型的数学本质都是拟合一个条件概率质量函数，拟合的目标函数可简单表示为如下形式：

$$P(y_t | y_1, \dots, y_{t-1})$$

即第 t 个token为 y_t 的概率是该token序列的前 $t - 1$ 个token的条件概率。对于每一个给定的长度为T的输入 X ,

$$P(X) = \prod_{t=1}^T P(x_t | x_{<t})$$

然后引入损失函数 $L(X)$ 来度量 $P(X)$ 的误差。训练阶段就是沿着负梯度方向不断调整模型参数，使损失函数 $L(X)$ 逐步下降到（局部）最优解，此时梯度趋近于 0，得到的这组参数就是最终的模型参数。需要注意的是，判断模型是否过拟合，应看训练损失与验证损失之间的差距（训练损失下降而验证损失上升），而非训练损失本身是否接近 0；语言模型的交叉熵损失在实际中几乎不会真正降到 0。

这种“预测下一个 token”的自回归目标无需人工标注，可直接从无标注文本语料中学习，因此属于自监督学习。

以上是对LLM模型的一个数学直观的理解，下面我们将按照 Transformer 算法从训练到推理的过程按步骤介绍 Transformer 各个环节的基本原理，但不会涉及模型训练和推理过程中的工程细节。

§ STEP 0 构建语料库与全局词表

在训练模型之前，我们需要为模型构建一个全局词表，这是模型理解文本的基础，可以将词表形象的理解成模型的全部词汇量。构建词表的方式通常是使用分词器（Tokenizer，分词器也是在语料库上通过统计方法拟合出来的）对预训练语料库进行分词，将其转换成一个如下形式的列表：

$$\begin{bmatrix} Token_1 : TokenID_1 \\ Token_2 : TokenID_2 \\ \vdots \\ Token_{V-1} : TokenID_{V-1} \\ Token_V : TokenID_V \end{bmatrix}$$

其中， $Token_i$ 表示词表中的第 i 个token的字符串， $TokenID_i$ 表示第 i 个token所对应的唯一编号。

词表构建完成后，大模型的 Token Embedding Layer 会创建一个 Embedding 矩阵 $C \in \mathbb{R}^{V \times d}$ ，并将词表映射到这个 Embedding 矩阵上，其行号对应词表中的 $TokenID$ （这是一个一一映射），因此 Embedding 矩阵的行数等于词表的 Token 总数。

Embedding 使得词表的每个 $Token_i$ 都映射到 Embedding 矩阵中的第 i 个行向量，其中行向量的维数为 d ，而 $Token_i$ 的语义正是保存在这个行向量中。Embedding 矩阵 C 是一个可训练矩阵，它的每个行向量的初始值都是随机生成的，此时词表中的Token之间的关系是混沌的，这表示模型在训练前对人类语言的语义“一无所

知”，需要在训练过程中，通过梯度下降不断更新矩阵 C 的每一行，才能使每个 token 的行向量逐渐学习到人类语言中的准确语义和语法信息，使得词表中的 Token 之间的关系逐渐明晰。

§ STEP 1 输入嵌入 (Embedding)：把文本信息映射到连续的向量空间

现在假设模型单次训练的输入是一个长度为 T 的 Token 序列 $X = (x_1, \dots, x_T)$ ，以下是 Embedding 阶段处理输入 X 的过程：

① $\forall x_i \in X$, 令 $x_i = \text{Lookup}(C, \text{TokenID}_i)$ ，即先在全局词表中找到与 x_i 相同的 Token_i ，然后根据 Token_i 对应的 TokenID_i 找到矩阵 C 的第 TokenID_i 行，并将该行向量赋值给 x_i 。这一步之后我们会得到一个矩阵

$$X = \begin{bmatrix} x_1 \\ \vdots \\ x_T \end{bmatrix}, X \in \mathbb{R}^{T \times d}$$

② 位置注入：将输入矩阵 X 转换为带位置信号的矩阵

因为Transformer的注意力模块本身不具备内置的位置先后感知能力（置换等变性），而人类语言中相同词汇在语句的不同位置，其含义往往不同，所以需要为 X 中的每一个 token x_i 显式注入一个位置信号（向量） p_i 。位置编码 p_i 的具体形式有多种：原始 Transformer 采用正余弦（sinusoidal）函数，现代大语言模型则更多使用可学习的位置编码或旋转位置编码（RoPE，属于相对位置编码，LLaMA 等模型采用）。即：

$$X = \begin{bmatrix} x_1 + p_1 \\ \vdots \\ x_T + p_T \end{bmatrix}, X \in \mathbb{R}^{T \times d}$$

Transformer的缩放点积注意力核心算子 $\text{Attention}(X) = \text{Softmax}(XW_Q \cdot (XW_K)^T) \cdot XW_V$ 关于输入序列的索引置换具有等变性（Permutation Equivariance）。即对于任意置换矩阵 $P \in \{0, 1\}^{T \times T}$ ，在不考虑位置编码时，注意力函数满足 $\text{Attention}(PX) = P \cdot \text{Attention}(X)$ ，这意味着模型对输入序列 X 的行顺序不具备天然的归纳偏置（Inductive Bias），无法仅凭自注意力机制本身区分 x_i 与 x_{i+1} 的相对先后关系。因此，必须通过显式注入位置向量 p_i 来打破这种置换对称性（Permutation Symmetry），赋予模型对序列顺序的感知能力。

§ STEP 2 缩放点积注意力

① 线性投影：引入三个独立可训练矩阵 $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ ，其中 d 是模型维度， $d_k = \frac{d}{h}$ 是多头注意力中注意力头的数量，对于单头注意力， $h = 1$ ，所以 $d_k = d$ 。然后我们计算输入矩阵 X 在 W_Q, W_K, W_V 上的投影：

$$Q = X \times W_Q, K = X \times W_K, V = X \times W_V$$

其中 $Q, K, V \in \mathbb{R}^{T \times d_k}$ 。

② 初始注意力分数：

$$S = Q \times K^T, \text{ 其中 } S \in \mathbb{R}^{T \times T},$$

对 S 进行缩放：

$$Scores = \frac{S}{\sqrt{d_k}} = \frac{Q \times K^T}{\sqrt{d_k}}, \text{ 其中 } Scores \in \mathbb{R}^{T \times T}$$

除以 $\sqrt{d_k}$ 的原因：当 d_k 增大时， Q 与 K 的内积 $Q \cdot K^T$ 的方差也随之增大，会使 Softmax 输出趋向饱和、梯度变小；除以 $\sqrt{d_k}$ 可把方差拉回到 1 左右，从而稳定训练。

③ 添加因果掩码：为了满足自回归条件 $P(x_t | x_{<t})$ 所以对于 $j > i$ 的分量 x_j 不参与计算，即

$$Scores_{i,j} = -\infty, (i < j)$$

也就是说 $Scores$ 的右上角元素都是 $-\infty$ 。

④ 概率归一化与加权提取

$$Attention(Q, K, V) = Softmax(Scores) \cdot V$$

$Softmax(z)$ 是一个概率归一化函数，其形式为：

$$Softmax(z) = (Softmax(z_1), \dots, Softmax(z_K))$$

其中 z 是一个 K 维向量，

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \text{ 且 } \sum_{i=1}^K Softmax(z_i) = 1$$

在Transformer中， $Softmax(Scores)$ 对 $Scores$ 矩阵的行向量进行概率归一化，将注意力分数 $Scores$ 的行向量压缩到 $(0, 1)$ 区间，即对于矩阵中的第 i 行的第 j 个元素的计算：

$$Softmax(z_{ij}) = \frac{e^{z_{ij}}}{\sum_{k=1}^T e^{z_{ik}}}$$

因为， $Softmax(Scores) \in \mathbb{R}^{T \times T}$, $V \in \mathbb{R}^{T \times d_k}$ ，所以 $Attention(Q, K, V) \in \mathbb{R}^{T \times d_k}$ 。

值得注意，注意力分数矩阵 $Scores \in \mathbb{R}^{T \times T}$ 意味着标准注意力的时间与空间复杂度均为 $O(T^2)$ ，这是模型处理长文本时的根本瓶颈，也是线性注意力、稀疏注意力等改进方向的研究出发点。

§ STEP 3 多头注意力机制

前面讨论的是单头注意力（即， $d_k = d$ 的情形），多头注意力就是将 $Token$ 向量的维度 d 拆分成 h 份（即 h 个头），此时 $d_k = \frac{d}{h}$ ，然后分别计算每个头的 $Attention$ （即对每个头重复 STEP2 中的单头注意力计算），

$$\begin{aligned} head_i &= Attention(X \cdot W_{Q_i}, X \cdot W_{K_i}, X \cdot W_{V_i}) \\ &= Softmax(Scores_i) \cdot V_i = Softmax\left(\frac{Q_i \times K_i^T}{\sqrt{d_k}}\right) \cdot V_i \end{aligned}$$

其中 $W_{Q_i}, W_{K_i}, W_{V_i} \in \mathbb{R}^{d \times d_k}$, $Q_i, K_i, V_i, head_i \in \mathbb{R}^{T \times d_k}$, $K_i^T \in \mathbb{R}^{d_k \times T}$, $Scores_i \in \mathbb{R}^{T \times T}$

最后将多个头的 $Attention$ 拼接成

$$MultiHead(X) = Concat(head_1, \dots, head_h) \cdot W_O$$

其中 $W_O \in \mathbb{R}^{d \times d}$ ，是一个可训练的输出投影矩阵， $Concat$ 表示将 $head_1, \dots, head_h$ 按顺序首尾相连拼接成一个矩阵，所以 $Concat(head_1, \dots, head_h) \in \mathbb{R}^{T \times d}$ ，因此 $MultiHead(X) \in \mathbb{R}^{T \times d}$ 。

多头的意义在于：将 d 维空间拆成 h 个独立的表示子空间并行计算注意力，使模型能同时捕捉多种不同类型的关联（如语法、指代、语义关系），表达能力明显强于单头。

§ STEP 4 多头注意力的残差连接与层归一化

① 残差链接（Residual Connection）：是指将前面的计算的多头注意力 $MultiHead(X)$ 作为残差添加到输入矩阵 X 上，即

$$Out = X + Sublayer(X)$$

， $Sublayer(X)$ 表示残差函数的声明，而不代表具体函数，在Transformer多头注意力中， $Sublayer(X) = MultiHead(X)$ ，后面在前馈神经网络的残差连接中的 $Sublayer(X)$ 则有所不同。

② 层归一化（LayerNorm）：

$$LN(x) = \gamma \odot \frac{x - \mu}{\delta} + \beta$$

，其中 γ 和 β 是可训练的缩放、偏移参数，而 x 就是残差链接得到的 Out 。

经过上面的残差链接和层归一化，我们得到了一个新矩阵：

$$X_1 = LN(X + MultiHead(X))$$

，显然 $X, X_1 \in \mathbb{R}^{T \times d}$ 。这里的层归一化放在残差连接之后，称为 Post-LN，是原始 Transformer 的形式；现代大模型（如 GPT-2/3、LLaMA）多改用 Pre-LN（即 $X + Sublayer(LN(X))$ ），训练更稳定，本文统一以 Post-LN 为例。

残差连接为梯度提供了直达底层的短路通道，可缓解深层网络的梯度消失；层归一化则稳定各层输入的分布，二者共同保障了深层 Transformer 的稳定训练。

§ STEP 5 前馈神经网络（FFN）

① 前馈神经网络的数学形式：

$$FFN(X) = GELU(XW_1 + b_1)W_2 + b_2$$

其中 $W_1 \in \mathbb{R}^{d \times 4d}$ ， $W_2 \in \mathbb{R}^{4d \times d}$ ，偏置向量 $b_1 \in \mathbb{R}^{4d}$ ， $b_2 \in \mathbb{R}^d$ 。这里的 W_1, W_2, b_1, b_2 都是可训练参数，GELU则是一个人为选择的不可训练的非线性函数。

② 将MultiHead传递到FFN中

- 残差链接： $X_2 = X_1 + FFN(X_1)$
- 层归一化： $X_{out} = LN(X_2) = LN(X_1 + FFN(X_1))$

到这里我们完成了一层 Transformer 运算，得到了输出 X_{out} 。这个 X_{out} 将作为下一层 Transformer 的输入 X ，重复前面的运算步骤，直到完成一次前向传播。一次前向传播通常会包含 L 层 Transformer，比如 GPT-3 是

96 层。每一层的输出可以记为 $X_{out}^1, X_{out}^2, \dots, X_{out}^L$ ，特别地，我们将第 L 层的输出记为 $X_{out}^L = H \in \mathbb{R}^{T \times d}$ 。

为便于整体把握，将一次前向传播中主要变量的维度流转汇总如下：

$$\begin{aligned} X \in \mathbb{R}^{T \times d} &\xrightarrow{W_Q, W_K, W_V} Q, K, V \in \mathbb{R}^{T \times d_k} \xrightarrow{\frac{QK^T}{\sqrt{d_k}}} Scores \in \mathbb{R}^{T \times T} \xrightarrow{Softmax(\cdot) \cdot V} head_i \in \mathbb{R}^{T \times d_k} \\ &\xrightarrow{Concat \cdot W_O} MultiHead \in \mathbb{R}^{T \times d} \xrightarrow{FFN} H \in \mathbb{R}^{T \times d} \xrightarrow{W_{out}} Logits \in \mathbb{R}^{T \times V} \end{aligned}$$

§ STEP 6 输出投影与损失函数

① 输出投影

$$Logits = H \cdot W_{out} + b_{out}$$

其中 $W_{out} \in \mathbb{R}^{d \times V}$ ， $b_{out} \in \mathbb{R}^V$ 都是可训练的参数，并且 W_{out} 通常与 Embedding 矩阵 C 绑定（即权重绑定 Weight Tying，令 $W_{out} = C^T$ ）：好处是省去一个 $V \times d$ 的独立输出矩阵、减少参数并起正则作用；代价是输入表示与输出分类被耦合、灵活性受限。因此并非所有模型都强制绑定，不少模型（如 BERT、部分多语言模型）选择让输出层独立训练，以更大的参数换取更强的容量与解耦能力。

如果 W_{out} 与 Embedding 矩阵 C 绑定，则 W_{out} 直接取 C 的转置，不再不需要为其单独初始化和训练一组参数，这样在输出投影层省去了 W_{out} 的 $V \times d$ 个权重

② 概率归一化

$$P(x_t) = Softmax(Logits_t)$$

③ 计算损失函数（损失函数是人工选择，比如常用的交叉熵函数 $\mathcal{L}$ ）

$$\mathcal{L} = -\log(P(y)) = -Logits_y + \log\left(\sum_{j=1}^V e^{Logits_j}\right)$$

④ 计算损失函数的梯度

$$\frac{\partial \mathcal{L}}{\partial Logits} = P - Onehot(y)$$

因为 $Logits = H \cdot W_{out} + b_{out}$ ，所以我们可以依次计算出

$$\frac{\partial \mathcal{L}}{\partial W_{out}}, \frac{\partial \mathcal{L}}{\partial b_{out}}, \frac{\partial \mathcal{L}}{\partial H}$$

，通过链式法则逐层回传到

$$\frac{\partial \mathcal{L}}{\partial W_Q}, \frac{\partial \mathcal{L}}{\partial W_K}, \frac{\partial \mathcal{L}}{\partial W_V}, \frac{\partial \mathcal{L}}{\partial W_1}, \frac{\partial \mathcal{L}}{\partial W_2}, \frac{\partial \mathcal{L}}{\partial b_1}, \frac{\partial \mathcal{L}}{\partial b_2}, \dots$$

§ STEP 7 完整的训练过程

① 批量输入

即 $X \in \mathbb{R}^{B \times T \times d}$ 。其中 B 表示批次大小，即一次输入 B 个语句，每个语句包含 T 个 Token，每个 token 的向量维数为 d ，当 $B = 1$ 时， $X \in \mathbb{R}^{T \times d}$ ，就是我们前面讨论的简化情形。

② 并行计算

并计算 B 个批次，即独立计算 $X_1, X_2, \dots, X_B$ 的 Transformer。

③ 完成一次训练迭代 (step)

一次前向传播需要经过 L 层 Transformer，每一层都重复第 1 小节到第 6 小节的单层运算。

④ 计算损失函数和梯度

每一个 Step 完成后，计算一次损失函数 $\mathcal{L}$ 和梯度，这里同样每个输入 $X_1, X_2, \dots, X_B$ 的 $\mathcal{L}$ 和梯度都独立计算。

⑤ 求均值

对 B 个矩阵 $X_1, \dots, X_B$ 的 $\mathcal{L}_1, \dots, \mathcal{L}_B$ 及其相关的梯度求平均值（或者通过别的方式）计算出一个 $\mathcal{L}$ 和一组梯度。

⑥ 优化

优化器拿到这一组梯度后，对所有参数进行一次优化，比如，令 $W_{Q,\text{new}} = W_{Q,\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W_{Q,\text{old}}}$

⑦ 遍历语料库。

即在经过 N 个 step 后，输入 X 已经遍历了语料库中的所有语句，此时我们说完成了一个 Epoch 的训练。

⑧ 验证

在验证集中，使用经过 1 个 Epoch 训练的模型参数进行推理，然后计算在验证集上的损失函数 $\mathcal{L}_{\text{验证集}}$ ，然后根据 $\mathcal{L}_{\text{验证集}}$ 决定是否停止训练（比如，在经过 N 个 Epoch 后， $\mathcal{L}_{\text{验证集}}$ 不再下降。）

§ STEP 8 推理

初始输入 $X_{\text{prompt}} \in \mathbb{R}^{1 \times T_p \times d}$

① 预填充

使用 X_{prompt} 执行一次前向传播，即执行 L 层 Transformer，然后缓存每一层的每个位置的 K 和 V 矩阵，即 KV cache。

② 自回归循环

（为便于理解，下面以单层视角简化描述；实际推理中 KV cache 按层缓存，每层独立计算各自的 Q、K、V。）
在最后一层 Transformer 中计算出来的 H 的最后一行， $X_{H,T}$ 乘以训练好的 W_Q 得到下一个 Token 的 Q ，即 $Q_{\text{new}} = X_{H,T} \cdot W_Q$ 。也就是说，输出的第一个 Token 的 Q 是提示词最后一个 Token 在最后一层 Transformer 输出的 H 的最后一行乘以训练好的 W_Q 。

K, V 则是把提示词的每个 Token 的 H 与训练好的 W_K, W_V 点乘，得到 $K_{\text{cache}}, V_{\text{cache}}$ 作为第一个输出的 K, V 。然后在此基础上依次计算新 Token 的 MultiHead ， H_{new} ，再然后计算出新 Token 在词表中的投

影

$$Logits_{new} = H_{new} \cdot W_{out} + b_{out}$$

，其中 W_{out}, b_{out} 是训练好的参数。

最后根据计算出的 $Logits_{new}$ 从词表中取出对应的 Token，在实际工程中，通常会先通过特定的采样方法从 $Logits$ 中取值，然后再查表找出最合适的 Token。

之后的步骤就是利用上一个输出 Token 的 H 的最后一行计算下一个新的 Token 的 Q ，同时把上一个输出 Token 的 V, K 缓存在 KV cache 中，然后逐步计算出下一个 Token，如此循环直至计算出 [EOS] Token 或者达到模型输出窗口的上限。

KV cache 的核心价值在于：生成新 token 时无需重新计算历史 token 的注意力，使单步生成的计算量由 $O(T^2)$ 降为 $O(T)$ 。此外，训练采用 teacher forcing，一次前向传播即可并行得到所有位置的预测与损失；而推理只能自回归地逐 token 生成，这正是两者流程的本质区别。

§ 附录：编码器与解码器

① 相同的前向传播路径： $X_{input} \rightarrow MultiHead \rightarrow Add \rightarrow Norm \rightarrow FFN$

② 自注意力层的唯一区别：

$$S_{encoder} = Q_{enc} \cdot K_{enc}^T \rightarrow Output_{enc} = Softmax(\frac{S_{encoder}}{\sqrt{d_K}}) \cdot V_{enc}$$

$$S_{decoder} = Q_{dec} \cdot K_{dec}^T + M \rightarrow Output_{dec} = Softmax(\frac{S_{decoder}}{\sqrt{d_K}}) \cdot V_{dec}$$

其中， $M \in \mathbb{R}^{T \times T}$ 即因果掩码， M 的右上角元素全是 $-\infty$ 。也就是说 Encoder 和 Decoder 唯一的区别就是在计算注意力分数时，Decoder 比 Encoder 多加了一个因果掩码矩阵，除此之外，其他计算过程都是一样的。

③ Encoder-Decoder 架构：交叉注意力层

1° Encoder 和 Decoder 先各自完成一次自注意力 + FFN 计算。

2° Decoder 在完成自注意力 + FFN 计算之后，会再计算一次交叉注意力计算，交叉注意力层的 $Q = Output_{dec} \cdot W_Q, K = H_{enc} \cdot W_K, V = H_{enc} \cdot W_V$